Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores

CE4303 — Principios de Sistemas Operativos

Proyecto 1

Calendarizador de Tiempo Real Heterogéneo

Documentación

Profesor: Luis Barboza Artavia

Integrantes:

Jose Pablo Fernández Rojas - 2022180283

Darío Garro Moya - 2023072470

Jarod David de la O Segura - 2018147298

José Andrés Solano Mora - 2018102753

Fecha de entrega: 27/10/2026

Índice

1. Introducción **3**

2. Arquitectura general **3**

3. Nodos **4**

3.1. Calendarizador local 4

3.2. Unidad de trabajo y calibración 5

3.3. Nodo Raspberry Pi Pico W 6

3.4. Nodo ESP32 6

3.5. Hardware 6

4. Host **7**

4.1. Procesos y comunicación entre ellos 7

4.2. Planificación y admisión 7

4.3. Modo manual 8

4.4. Caída de nodos 8

4.5. Reporte 8

4.6. Configuración 9

5. Protocolo de comunicación **9**

6. Sincronización y desfase entre nodos **11**

7. Experimentos **12**

7.1. Entorno 12

7.2. First-Fit contra Worst-Fit 12

7.3. RMS contra EDF 14

7.4. Tareas esporádicas 15

7.5. Caída de nodos 16

7.6. Rechazos y entradas inválidas 16

7.7. Modo manual 16

7.8. Resumen de verificación 17

8. Compilación, carga y pruebas **17**

8.1. Host 17

8.2. Nodo Raspberry Pi Pico W 17

8.3. Nodo ESP32 17

8.4. Pruebas sin hardware 17

8.5. Experimentos y figuras 18

9. Atributo: Trabajo en Equipo **18**

10.Conclusiones **20**

A. Archivos de tareas de ejemplo **20**

1. Introducción

Este documento describe el diseño, la implementación y las pruebas de un sistema de calendarización de tiempo real distribuido en el que una computadora con Linux (el *host*) reparte un conjunto de tareas periódicas y esporádicas entre dos microcontroladores de distinta capacidad: un **ESP32** y una **Raspberry Pi Pico W**. Cada nodo ejecuta sus tareas con un calendarizador local propio que implementa **RMS** (*Rate Monotonic Scheduling*) y **EDF** (*Earliest Deadline First*) [1, 2]; el algoritmo se elige en la configuración al desplegar las tareas.

El host calibra los nodos, decide en qué nodo se ejecuta cada tarea (particionado con First-Fit o Worst-Fit), comprueba que cada nodo sea calendarizable, vigila que los nodos sigan vivos y genera el reporte final. El host **no** participa en la calendarización en tiempo de ejecución: cada nodo decide por su cuenta qué tarea avanza en cada tick y sigue funcionando aunque el host se desconecte.

El documento incluye, además de la descripción del sistema, el formato del protocolo de comunicación, la medición del desfase entre nodos, los experimentos pedidos (RMS contra EDF y First-Fit contra Worst-Fit) con resultados medidos sobre el hardware real, las instrucciones de compilación y carga, y el documento del atributo de Trabajo en Equipo.

2. Arquitectura general

La figura 1 muestra los componentes del sistema. El host está formado por cuatro procesos que se comunican por memoria compartida POSIX y semáforos POSIX. Cada nodo se conecta al host por USB y usa un protocolo serial con tramas verificadas por CRC-16.

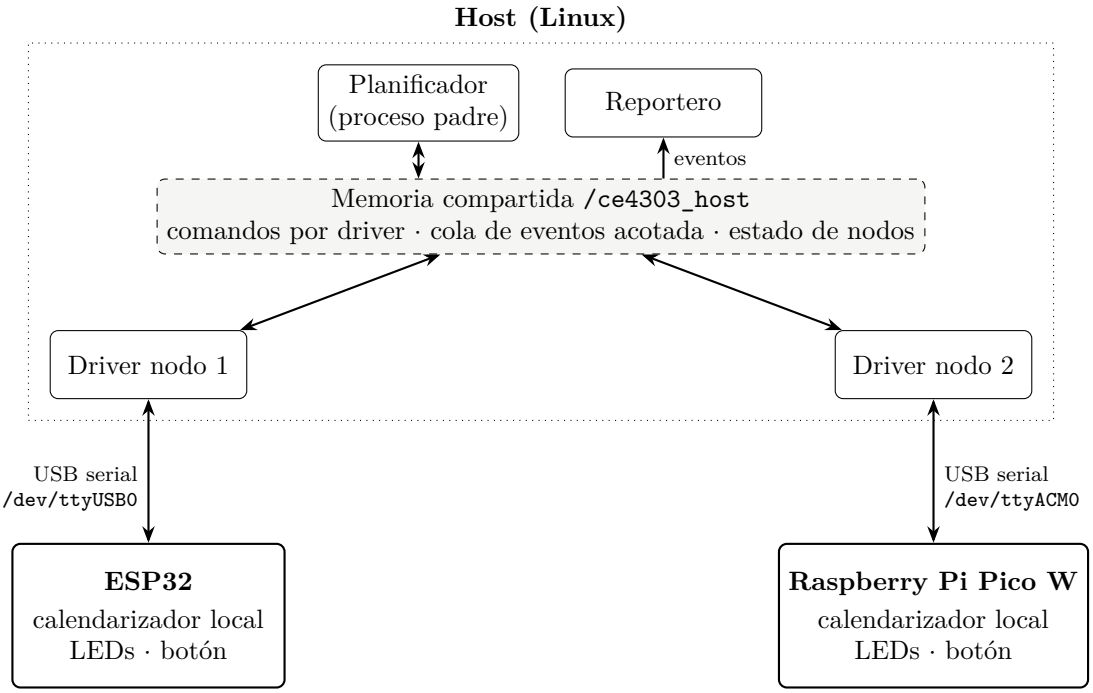


Figura 1: Arquitectura del sistema.

El repositorio se organiza así:

Carpeta	Contenido
`common/`	Protocolo compartido por host y nodos: armado y validación de tramas, CRC-16, formato de los mensajes (`wire_format.h`) y estructura de una tarea.
`firmware/core/`	Calendarizador local (`rts_engine`) y la unidad de trabajo (`ut.c`, sin modificar). Es C puro, sin código de hardware, y lo comparten los dos nodos.
`firmware/pico/`	Firmware de la Pico W (Pico SDK, CMake).
`firmware/esp32/`	Firmware del ESP32 (PlatformIO, framework Arduino).
`host/`	Host en C para Linux: planificador, drivers, reportero, IPC y configuración.
`tools/`	Pruebas y herramientas: pruebas del calendarizador y del planificador en PC, pruebas de integración, host falso, nodo de PC y generación de figuras.

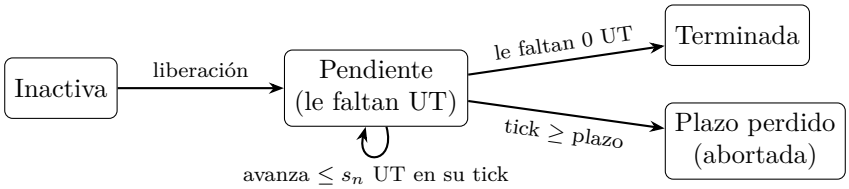
Separar el calendarizador (`firmware/core`) del código de cada placa permite escribirlo una sola vez para los dos nodos y probarlo en la PC sin hardware (sección 8.4).

3. Nodos

3.1. Calendarizador local

El calendarizador (`firmware/core/rts_engine.c`) mantiene, para cada tarea, su definición (id, tipo, W, T, D, criticidad) y el estado de su trabajo actual: si hay un trabajo pendiente, cuántas unidades de trabajo (UT) le faltan, en qué tick se liberó, su plazo absoluto y la siguiente liberación. Cada nodo soporta hasta **16 tareas** simultáneas (el enunciado pide al menos 8) y como máximo una esporádica.

Trabajo por tick (máquina de estados). Las tareas no tienen código propio: una tarea con trabajo W consiste en llamar W veces a `unidad_trabajo()`. Para que la apropiación ocurra al final de cualquier tick, cada trabajo avanza a lo sumo s_n UT por tick y guarda cuánto le falta (`remaining_ut`). Así, cada tarea es una máquina de estados sencilla:



Ciclo de un tick. En cada tick la plataforma llama a `rts_tick()` y luego a `rts_run()`:

1. **Plazos vencidos.** Todo trabajo pendiente cuyo plazo absoluto ya llegó se aborta, se cuenta y se reporta (evento de plazo perdido, LED rojo). Las demás tareas siguen normalmente.
2. **Liberaciones.** Las periódicas se liberan cada T ticks. La esporádica se libera si el botón la marcó y ya pasó su intervalo mínimo T; si no, la pulsación se ignora y se cuenta. El plazo absoluto del trabajo es *tick de liberación* $+D$.
3. **Selección.** Entre las tareas con trabajo pendiente se elige la de mayor prioridad: con RMS, la de menor período T (prioridad fija); con EDF, la de plazo absoluto más cercano.
4. **Cambio de tarea.** Si la tarea elegida cambió respecto al tick anterior se genera un evento de cambio.

5. **Ejecución** (`rts_run`). La tarea elegida llama a `unidad_trabajo()` s_n veces, o menos si le quedan menos UT. Si ya no le quedan, el trabajo termina al final de ese tick.

Criterio de desempate. Se usa el mismo criterio con RMS y con EDF, y se mantiene durante toda la ejecución: si dos tareas empatan en período (RMS) o en plazo absoluto (EDF), gana la de **mayor criticidad**; si también empatan, gana la de **menor id**.

Liberación con el trabajo anterior pendiente. Si llega la liberación de un trabajo nuevo y el anterior de la misma tarea sigue pendiente (solo puede pasar si $D > T$), el anterior se cuenta como plazo perdido y se descarta.

Eventos. El calendarizador no conoce el hardware ni el protocolo: genera eventos (liberación, fin, plazo perdido, pulsación ignorada, cambio de tarea) y los entrega a una función que le pasa la plataforma. Cada firmware traduce esos eventos a mensajes del protocolo y los deja en un búfer de salida.

3.2. Unidad de trabajo y calibración

Al recibir la solicitud de calibración (que trae la duración del tick), el nodo mide cuántas veces cabe `unidad_trabajo()` en un tick:

- Se hacen 20 mediciones y se toma la **peor** (la menor cantidad), porque las interrupciones del USB pueden robar tiempo en algunas.
- Cada medición dura como máximo 10 ms y el resultado se escala a la duración real del tick. Así, con ticks largos (por ejemplo 500 ms para ver los LEDs) la calibración no tarda más de lo que el host espera.
- Se usa solo el **70 %** del tick: el resto queda como margen para el calendarizador, los LEDs y el envío de eventos.

Las mediciones se repiten en cada ejecución y no se fijan en el código. Con un tick de 10 ms se obtuvo:

Nodo	**Reloj**	**s_n (UT/tick)**	**Tiempo por UT (aprox.)**
Raspberry Pi Pico W (RP2040)	125 MHz	86 – 87	81 μs
ESP32 (DevKit)	240 MHz	42	167 μs

Tabla 1: Capacidad medida de cada nodo con tick de 10 ms y margen del 70 %.

Aunque el ESP32 tiene un reloj mayor, en nuestras mediciones la Pico ejecutó cada UT en menos de la mitad del tiempo. Atribuimos la diferencia a cómo se compila y se ejecuta el código en cada plataforma (por ejemplo, el nivel de optimización de PlatformIO y la ejecución desde memoria flash del ESP32); no afecta el funcionamiento porque el host usa el valor medido. Con la Pico se verificó que el trabajo de un tick ocupa como máximo el 69–71 % del tick (6.9–7.1 ms de 10 ms) y que no hubo ningún tick atrasado.

Con estas capacidades, el costo de una tarea en ticks es distinto en cada nodo, $C(i, n) = \lceil W(i)/s_n \rceil$. Por ejemplo, $W = 300$ cuesta 4 ticks en la Pico y 8 en el ESP32.

3.3. Nodo Raspberry Pi Pico W

Firmware en C con el Pico SDK (`firmware/pico`) [4].

- **Tick:** un temporizador de hardware (`add_repeating_timer_us`) cuya interrupción solo incrementa un contador. El lazo principal duerme con `__wfi()` hasta la siguiente interrupción y procesa cada tick pendiente. No se usa `sleep` ni espera activa para temporizar.
- **Botón:** interrupción por flanco de bajada con antirrebote de 200 ms; la rutina solo marca la liberación (el calendarizador hace el resto).
- **Comunicación** (`link.c`): la recepción arma las tramas byte por byte con una máquina de estados, sin bloquear, y descarta una trama incompleta después de 100 ms. El envío usa un **búfer circular** de 2 KiB: las tramas se copian ahí y se transmiten en segundo plano, solo lo que cabe en el USB en ese momento, así que el envío nunca atrasa un tick. Si el host no está conectado, el calendarizador sigue igual.

3.4. Nodo ESP32

Firmware en C++ con PlatformIO y el framework Arduino (`firmware/esp32`) [5]. Usa el mismo calendarizador (`lib/core` es un enlace a `firmware/core`).

- **Tick:** temporizador de hardware a 1 MHz cuya interrupción incrementa un contador y despierta la tarea principal con una notificación. FreeRTOS solo se usa para despertar; el orden de ejecución lo decide el calendarizador propio.
- **Botón:** interrupción que solo marca el flanco; el antirrebote (50 ms) se resuelve fuera de la interrupción.
- **Comunicación:** búfer de transmisión de 1 KiB, de modo que enviar eventos no bloquea el tick.

3.5. Hardware

Cada nodo tiene cuatro LEDs que muestran en binario el id de la tarea en ejecución (apagados cuando no corre ninguna), un LED de plazo perdido (se mantiene encendido 300 ms) y un botón para la tarea esporádica. Cada LED lleva una resistencia de 220–330 Ω; el botón va a GND y usa la resistencia de pull-up interna.

Función	Pico W (GPIO / pin)	ESP32 (GPIO)
LED tarea, bit 0	GP2 / 4	32
LED tarea, bit 1	GP3 / 5	33
LED tarea, bit 2	GP4 / 6	26
LED tarea, bit 3	GP5 / 7	25
LED plazo perdido	GP6 / 9	23
Botón (a GND)	GP15 / 20	22

Tabla 2: Conexiones de cada nodo.

4. Host

4.1. Procesos y comunicación entre ellos

El host (`host_app`) está formado por cuatro procesos:

- **Planificador** (proceso padre): lee la configuración y las tareas, pide la calibración, hace la admisión y la asignación, atiende el teclado (modo manual y la tecla `q`) y vigila la caída de nodos.
- **Un driver por nodo**: es el único proceso que lee y escribe el puerto serial de su nodo. Ejecuta los comandos del planificador (calibrar, configurar, desplegar, iniciar, detener) y mete en la cola cada evento que llega del nodo.
- **Reportero**: consume la cola de eventos y genera el reporte.

Se comunican por un objeto de memoria compartida POSIX [7] (`shm_open` y `mmap`) con semáforos POSIX. La cola de eventos entre los drivers (productores) y el reportero (consumidor) es **acotada** (4096 eventos) y sincronizada con un semáforo de exclusión mutua y dos semáforos de conteo (espacios libres y eventos disponibles). Cada driver recibe sus comandos por una casilla propia con sus semáforos. Ningún proceso hace espera activa: se espera en semáforos o en `poll()` con tiempo límite. Al terminar se detienen los nodos, se cierran los puertos, se espera a los procesos hijos y se liberan la memoria compartida y los semáforos.

4.2. Planificación y admisión

Con las capacidades s_n medidas, el costo de la tarea i en el nodo n es $C(i,n) = \lceil W(i)/s_n \rceil$ y su utilización es $C(i,n)/T(i)$. La tarea esporádica se trata como periódica con período igual a su intervalo mínimo.

1. Las tareas se ordenan de mayor a menor utilización medida en el nodo de mayor s_n (desempate: mayor criticidad y luego menor id).
2. Cada tarea se asigna con la heurística configurada [3]:
   - **First-Fit:** se recorren los nodos en orden decreciente de s_n y se elige el primero donde pasa la admisión.
   - **Worst-Fit:** entre los nodos donde pasa la admisión se elige el que queda con más capacidad libre después de asignarla (si empatan, el de mayor s_n).

3. **Prueba de admisión** por nodo, con las tareas que ya tiene más la nueva:

$$\text{EDF: } \sum_i \frac{C_i}{T_i} \leq 1 \qquad \text{RMS: } \sum_i \frac{C_i}{T_i} \leq n\left(2^{1/n} - 1\right)$$

   El límite de RMS es el de Liu y Layland [1]. Además, el nodo no puede pasar de 16 tareas ni tener más de una esporádica.

4. Si la tarea no cabe en ningún nodo, se **rechaza** y se informa el motivo; el resto se sigue asignando.

4.3. Modo manual

Mientras corre, el usuario puede escribir `a ID P/S W T D criticidad`. El planificador repite la admisión con el mismo algoritmo y la misma heurística (sin considerar nodos caídos) y, si procede, despliega la tarea sin detener el sistema; el nodo la empieza a liberar en el tick en que la recibe.

4.4. Caída de nodos

Cada nodo envía un reporte de tick cada N ticks (`reporte_cada`). Si el host deja de recibir K reportes seguidos (`reportes_perdidos`), es decir, si pasan más de $N \cdot K \cdot$ tick sin reportes, o si el puerto se desconecta, declara el nodo caído y:

1. Las tareas del nodo que sobrevive se mantienen.
2. Las periódicas del nodo caído se prueban en el nodo vivo de mayor a menor criticidad, con el mismo algoritmo y la misma prueba de admisión; cada una que pasa se suma al nodo antes de probar la siguiente. Así, si no caben todas, se rechazan primero las de menor criticidad.
3. Las tareas reubicadas empiezan de cero en el nodo nuevo.
4. Las esporádicas no se reubican y se reportan como rechazadas.

4.5. Reporte

Al presionar `q` el host detiene los nodos, termina los procesos, libera los recursos y genera en la carpeta configurada:

- `resumen.txt`: por nodo, la capacidad, la utilización teórica y la medida, los cambios de tarea, las caídas y las estadísticas por tarea (ticks ejecutados, liberaciones, finalizaciones, plazos perdidos, esporádicas, pulsaciones ignoradas y latencia de respuesta); además, las tareas aceptadas y rechazadas.
- `eventos.csv`: todos los eventos (liberaciones, ejecución, plazos, fallas) y los rechazos con su motivo.
- `gantt.csv` y `gantt.svg`: la tarea que corrió en cada tick de cada nodo (diagrama de Gantt).

4.6. Configuración

Clave	Ejemplo	Significado
`algoritmo`	EDF	`RMS` o `EDF`
`heuristica`	FF	`FF` (First-Fit) o `WF` (Worst-Fit)
`tick_ms`	10	Duración del tick en milisegundos
`reporte_cada`	5	Ticks entre reportes de tick (N)
`reportes_perdidos`	4	Reportes seguidos sin llegar para declarar un nodo caído (K)
`puerto_esp32`	/dev/ttyUSB0	Puerto serial del nodo 1
`puerto_pico`	/dev/ttyACM0	Puerto serial del nodo 2
`archivo_tareas`	host/config/...	Archivo de tareas (`id,tipo,W,T,D,criticidad`)
`carpeta_reporte`	output	Carpeta donde se guarda el reporte

Tabla 3: Archivo de configuración `host/config/host.conf`.

5. Protocolo de comunicación

La comunicación host–nodo usa el puerto serial USB a 115 200 baudios, 8N1. Todas las tramas tienen el mismo formato:

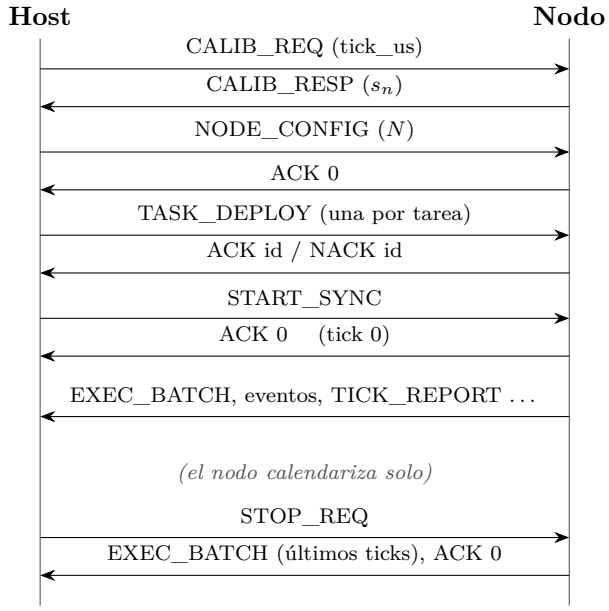
Inicio	Tipo	Largo	Carga útil	CRC-16
`0xAA`	1 B	1 B	0–255 B	2 B

- **Largo** es la cantidad de bytes de la carga útil.
- **CRC-16/MODBUS** [6] (polinomio `0x8005` reflejado, valor inicial `0xFFFF`) calculado sobre inicio, tipo, largo y carga útil; se envía en little-endian.
- Todos los enteros de la carga útil van en **little-endian**, campo por campo (no se envían estructuras de C, para evitar diferencias de relleno entre plataformas).
- **Tramas corruptas:** quien recibe una trama con CRC incorrecto la descarta y la reporta. El nodo responde `NACK`; el host la registra como `TRAMA_INVALIDA` en el reporte. Una trama que queda incompleta también se descarta.
- Los eventos llevan la marca de tiempo en ticks desde el mensaje de inicio. En los mensajes de ejecución el id 0 significa “ninguna tarea”.

La secuencia normal es la siguiente (el modo manual y la reubicación usan `TASK_DEPLOY` con el sistema en marcha):

Tipo	**Sentido**	**Carga útil**	**Uso / respuesta**
`0x01 CALIB_REQ`	H → N	tick_us[4]	Duración del tick y solicitud de calibración. Responde `CALIB_RESP`.
`0x02 CALIB_RESP`	N → H	ut_por_tick[4]	Capacidad medida s_n.
`0x10 NODE_CONFIG`	H → N	N_reporte[2]	Cada cuántos ticks enviar el reporte de tick. Responde `ACK` 0.
`0x03 TASK_DEPLOY`	H → N	alg[1] id[1] tipo[1] W[4] T[4] D[4] crit[1]	Despliegue de una tarea (16 B; alg: 0 = RMS, 1 = EDF; tipo: `'P'` o `'S'`). Responde `ACK` id o `NACK` id.
`0x05 START_SYNC`	H → N	—	Inicio: establece el tick 0. Responde `ACK` 0.
`0x0F TRIGGER_TASK`	H → N	id[1]	Disparo de la esporádica desde el host. Responde `ACK` 0.
`0x09 STOP_REQ`	H → N	—	Detención. Responde `ACK` 0.
`0x04 ACK`	N → H	id[1]	Acuse de recibo (id de la tarea o 0).
`0xFF NACK`	ambos	id[1]	Trama corrupta, mensaje inválido o tarea rechazada por el nodo.
`0x06 TICK_REPORT`	N → H	tick[4] tarea[1]	Reporte de tick cada N ticks (el "sigo vivo").
`0x0C EXEC_BATCH`	N → H	tick_inicial[4] cantidad[1] ids[cantidad]	Tarea que corrió en cada tick (Gantt).
`0x0A EVENT_RELEASE`	N → H	tick[4] id[1]	Liberación de un trabajo.
`0x07 EVENT_SPORADIC`	N → H	tick[4] id[1]	Liberación de la esporádica (para medir su latencia).
`0x0B EVENT_FINISH`	N → H	tick[4] id[1]	Fin de un trabajo.
`0x08 EVENT_DEADLINE`	N → H	tick[4] id[1]	Plazo perdido (trabajo abortado).
`0x0D SPORADIC_SKIP`	N → H	tick[4] id[1]	Pulsación ignorada (antes del intervalo mínimo).
`0x0E EVENT_START`	N → H	tick[4] id[1]	Inicio de ejecución de una tarea (solo ESP32; informativo).

Tabla 4: Mensajes del protocolo (definidos en `common/include/wire_format.h`).



6. Sincronización y desfase entre nodos

El mensaje `START_SYNC` establece el tick 0 en los dos nodos. El host envía el inicio a los dos nodos antes de esperar las respuestas, de modo que ambos lo reciben casi al mismo tiempo. Medimos el desfase de dos formas:

- **Diferencia entre los `ACK` de inicio.** En todas las ejecuciones con las dos placas fue de **1 a 3 ms**, menos de un tick (10 ms). Es una cota del desfase de arranque: incluye el tiempo de transmisión y de procesamiento del USB de cada nodo.
- **Ticks reportados por cada nodo al final.** Si los dos nodos avanzan al mismo ritmo, al detenerlos deberían haber reportado casi la misma cantidad de ticks.

Ejecución	Ticks ESP32	Ticks Pico	Diferencia
`tasks_input`, FF (≈148 s)	14 825	14 826	1
`tasks_input`, WF (≈146 s)	14 610	14 615	5
`tareas_rms_vs_edf`, EDF + FF (15 s)	1 460	1 460	0
`tareas_rms_vs_edf`, EDF + WF (15 s)	1 458	1 460	2
`tareas_rms_vs_edf`, RMS + FF (15 s)	1 460	1 461	1
`tareas_rms_vs_edf`, RMS + WF (15 s)	1 460	1 461	1

Tabla 5: Ticks de Gantt reportados por cada nodo al detenerlos.

En ejecuciones cortas la diferencia es de 0 a 2 ticks, y en ejecuciones de más de dos minutos se mantiene en pocos ticks (hasta 5, es decir 50 ms en ≈146 s, del orden de 0.03 %). Esa diferencia combina dos cosas: que el mensaje de detención tampoco llega exactamente al mismo tiempo a los dos nodos, y la pequeña diferencia entre los osciladores de cada placa, que hace que un nodo avance unos ticks más que el otro en ejecuciones largas. Como cada nodo calendariza de forma

independiente, este desfase no afecta el cumplimiento de plazos; solo hace que los tiempos de un nodo y otro en el Gantt no sean exactamente comparables tick por tick.

7. Experimentos

7.1. Entorno

Todas las pruebas de esta sección se hicieron con las dos placas reales conectadas al host: ESP32 como nodo 1 ($s_1 = 42$) y Pico W como nodo 2 ($s_2 = 87$), tick de 10 ms, $N = 5$ y $K = 4$. Para los experimentos se diseñó la carga `host/config/tareas_rms_vs_edf.txt`, con valores de W elegidos para que el costo en ticks sea el mismo en todo el rango de s_n medido entre ejecuciones (41–43 y 85–89):

						Pico ($s = 87$)		**ESP32** ($s = 42$)	
Id	**Tipo**	**W**	**T**	**D**	**Crit.**	C	U	C	U
1	P	320	10	10	5	4	0.40	8	0.80
2	P	150	10	10	4	2	0.20	4	0.40
3	P	230	15	15	3	3	0.20	6	0.40
4	P	320	20	20	2	4	0.20	8	0.40
5	P	150	10	10	1	2	0.20	4	0.40
6	S	80	20	20	3	1	0.05	2	0.10
7	P	150	20	20	2	2	0.10	4	0.20

Tabla 6: Carga de los experimentos (`tareas_rms_vs_edf.txt`). La esporádica se cuenta con su intervalo mínimo como período.

La utilización total en la Pico es 1.35, así que la carga no cabe en un solo nodo y obliga a repartir. Cada combinación se corrió con `tools/correr_experimentos.sh`, que guarda el reporte de cada una en `resultados/`.

7.2. First-Fit contra Worst-Fit

Configuración	**Pico (nodo 2)**		**ESP32 (nodo 1)**		**Rechazadas**
EDF + FF	1, 2, 3, 4	($U = 1.00$)	5, 6, 7	($U = 0.70$)	—
EDF + WF	1, 3, 4, 7	($U = 0.90$)	2, 5, 6	($U = 0.90$)	—
RMS + FF	1, 2, 6, 7	($U = 0.75$)	3, 4	($U = 0.80$)	5
RMS + WF	1, 3, 6, 7	($U = 0.75$)	2, 4	($U = 0.80$)	5

Tabla 7: Asignación obtenida en cada combinación (utilización teórica de cada nodo).

Con la misma carga, las dos heurísticas producen asignaciones diferentes con ambos algoritmos:

- **First-Fit** llena primero el nodo más rápido: con EDF la Pico queda exactamente al 100 % y el ESP32 recibe solo lo que ya no cabe.

- **Worst-Fit** reparte la carga: con EDF ambos nodos quedan al 90 %. Por ejemplo, la tarea 2 va a la Pico con FF (es el primer nodo donde cabe) y al ESP32 con WF (ahí queda más espacio libre: 0.6 en el ESP32 contra 0.4 en la Pico).

En todas las combinaciones **no se perdió ningún plazo** y la utilización medida coincidió con la teórica. En EDF + FF la Pico trabajó con **utilización medida de 1.0000** sin perder plazos. Las diferencias que se ven en el ESP32 (por ejemplo 0.60 medida contra 0.70 teórica en EDF + FF) se deben a la esporádica: la teoría la cuenta en su peor caso (una liberación cada 20 ticks), pero en la prueba solo se libera cuando se presiona el botón.

	Pico		**ESP32**	
Configuración	teórica	medida	teórica	medida
EDF + FF	1.000	1.000	0.700	0.600
EDF + WF	0.900	0.901	0.900	0.801
RMS + FF	0.750	0.700	0.800	0.802
RMS + WF	0.750	0.702	0.800	0.800

Tabla 8: Utilización teórica y medida por nodo en las corridas de 15 s de `correr_experimentos.sh` (sin presionar el botón de la esporádica).

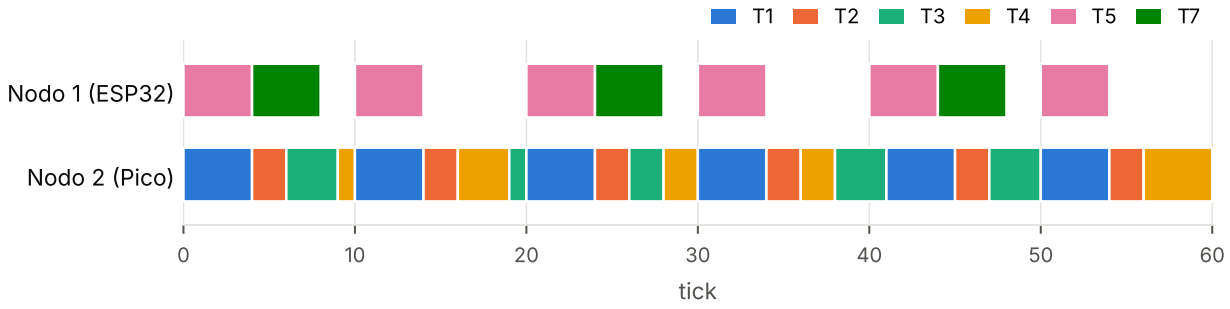


Figura 2: Gantt de los primeros 60 ticks con EDF + First-Fit.

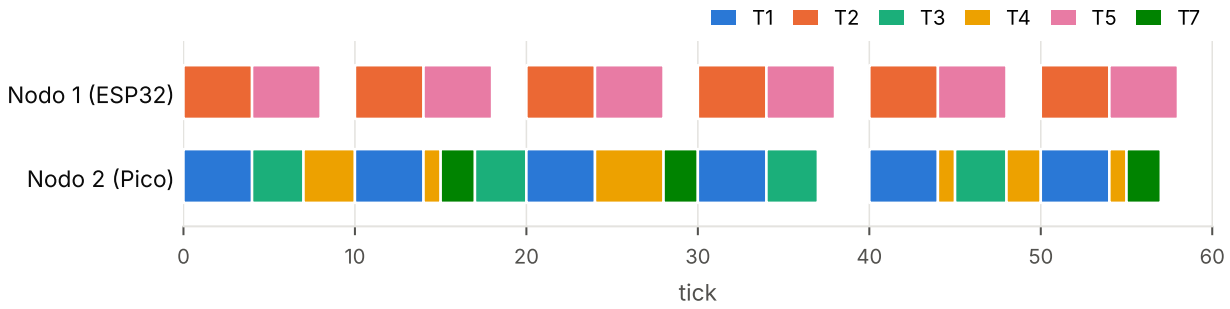


Figura 3: Gantt de los primeros 60 ticks con EDF + Worst-Fit.

7.3. RMS contra EDF

El experimento se hizo en dos niveles.

Admisión (host). Con la misma carga, EDF acepta las 7 tareas y RMS rechaza la tarea 5 con las dos heurísticas (tabla 7). Con RMS el límite de un nodo depende de la cantidad de tareas [1]: 0.828 con 2, 0.780 con 3 y 0.757 con 4. La tarea 5 ($U = 0.2$ en la Pico, 0.4 en el ESP32) ya no cabe bajo ese límite en ningún nodo, mientras que con EDF basta con que la suma no pase de 1. Todo lo que RMS admitió se ejecutó sin perder plazos: la prueba de admisión garantiza los plazos de las tareas aceptadas.

Ejecución con una carga que solo cumple EDF. Para ver qué pasa cuando RMS ejecuta una carga que pasa su límite, se le envía directamente a un nodo (con `tools/fake_host.py`, sin pasar por la admisión del host) la carga T_1: $C = 2$, $T = D = 5$ y T_2: $C = 4$, $T = D = 7$, con $U = 2/5 + 4/7 = 0.971$. Esta carga cumple con EDF ($U \leq 1$) pero pasa el límite de RMS para dos tareas (0.828).

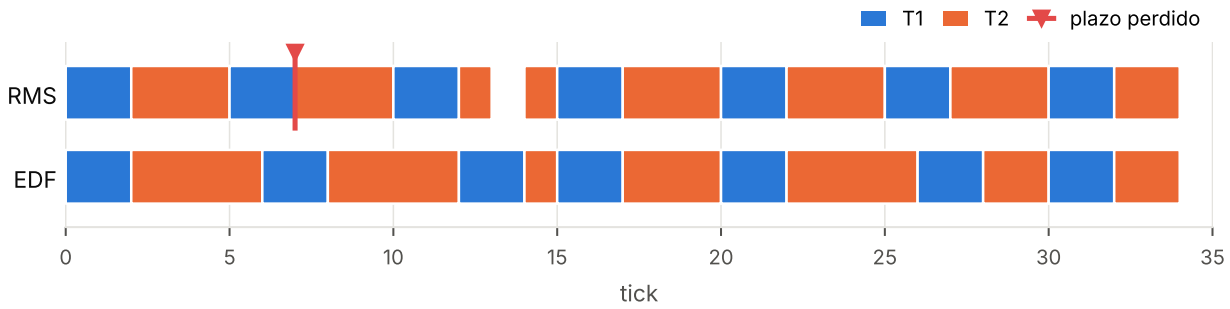


Figura 4: Los primeros 35 ticks de la carga T_1 (2/5) y T_2 (4/7) con RMS y con EDF (calendarizador ejecutado en la PC con `tools/test_rts.c`).

Con RMS, T_1 (período menor) siempre tiene prioridad. En el tick 5 se libera su segundo trabajo y desplaza a T_2, que solo alcanza a avanzar 3 de sus 4 ticks antes de su plazo (tick 7), así que **pierde el plazo** y se aborta. Con EDF, en el tick 5 el plazo de T_2 (7) es más cercano que el del nuevo trabajo de T_1 (10), así que T_2 sigue, termina a tiempo y **no se pierde ningún plazo** en todo el hiperperíodo de 35 ticks.

Entorno	RMS	EDF
Calendarizador en la PC (`test_rts`, 35 ticks)	1 plazo perdido	0
Nodo de PC con el firmware de la Pico (8 s)	23 plazos perdidos	0
Pico W real (8 s)	23 plazos perdidos	0

Tabla 9: Plazos perdidos de T_2 con la carga que solo cumple EDF.

En la Pico W real, con RMS, T_2 perdió su plazo exactamente una vez cada 35 ticks (en los ticks 7, 42, 77, 112, ...), como predice la simulación: 23 plazos perdidos en 8 segundos. Con EDF no se perdió ninguno. Además, la secuencia de tareas que reportó la placa tick por tick es idéntica a la de

la figura 4 (con RMS, `1122211222112.2...`; con EDF, `11222211222211211222112222...`), lo que confirma que el calendarizador se comporta igual en la placa que en la PC.

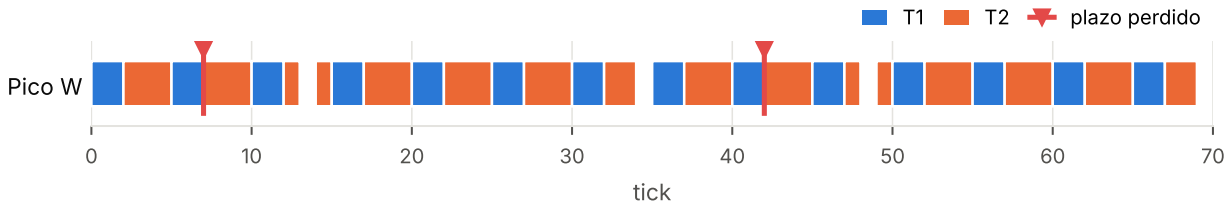


Figura 5: Ejecución en la Pico W con RMS (primeros 70 ticks).

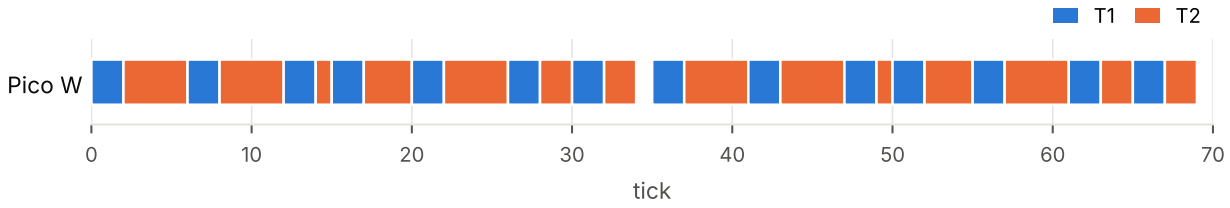


Figura 6: Ejecución en la Pico W con EDF (primeros 70 ticks).

7.4. Tareas esporádicas

Las esporádicas se liberaron con el botón de cada placa y también con el mensaje `TRIGGER_TASK`. En todas las pruebas se respetó el intervalo mínimo: las pulsaciones que llegaron antes se ignoraron y se contaron. La latencia de respuesta se mide desde la liberación hasta el primer tick en que la tarea se ejecuta.

Ejecución	**Nodo**	**Liberadas**	**Ignoradas**	**Latencia media**
`tasks_input`, FF	Pico	7	3	0.00 ticks
`tasks_input`, WF	ESP32	10	6	0.00 ticks
`tasks_input`, WF con caída	ESP32	30	38	0.12 ticks
`tareas_rms_vs_edf`, EDF + WF	ESP32 (al 80 %)	1	0	7.00 ticks

Tabla 10: Latencia de respuesta de las tareas esporádicas.

Con poca carga, la esporádica empieza a ejecutarse en el mismo tick en que se libera. Con el ESP32 al 80 % de utilización, la latencia subió a 7 ticks: con EDF, la esporádica (plazo de 20 ticks) espera a que terminen los trabajos con plazo más cercano, y aun así cumple su plazo.

Plazo perdido con plazo menor al período. La prueba de admisión del enunciado solo mira C/T, es decir, supone $D = T$. Para mostrar el LED de plazo perdido se usó `tareas_miss_esporadica.txt`, donde la esporádica de cada nodo tiene $D = 1$ tick pero cuesta 2 ticks en la Pico. La admisión la acepta ($C/T = 2/50$), pero cada vez que se presiona el botón el trabajo no alcanza a terminar: en la prueba hubo 3 liberaciones, 0 finalizaciones y **3 plazos**

perdidos, con el LED rojo encendido en cada uno, mientras que las periódicas del mismo nodo no perdieron ningún plazo (el calendarizador aborta solo el trabajo vencido). Esto muestra una limitación de la prueba de admisión por utilización: solo garantiza los plazos cuando $D = T$ [2].

7.5. Caída de nodos

Se probaron dos escenarios desconectando el cable USB de una placa con el sistema en marcha.

El nodo sobreviviente tiene espacio. Con `tasks_input.txt` y Worst-Fit (Pico: tareas 1 y 3; ESP32: tareas 2 y 4) se desconectó la Pico después de 875 ticks. El host la declaró caída y reubicó las tareas 1 y 3 en el ESP32, que ya tenía la 2 y la 4 y las mantuvo. Desde ese momento el ESP32 ejecutó las cuatro tareas sin perder plazos (por ejemplo, 275 liberaciones de la tarea 1), y el reporte registra 1 caída y 6 confirmaciones de despliegue (4 iniciales y 2 reubicadas).

El nodo sobreviviente no tiene espacio. Con `tareas_rms_vs_edf.txt`, EDF y Worst-Fit, los dos nodos estaban al 90 %. Al desconectar el ESP32, el host probó sus periódicas en la Pico de mayor a menor criticidad (primero la 2 y luego la 5), pero cualquiera la llevaba al 110 %, así que las **rechazó**; la esporádica 6 se rechazó porque las esporádicas no se reubican. Las tareas de la Pico siguieron sin cambios. Después se desconectó también la Pico: como ya no quedaba ningún nodo, se rechazaron sus tareas y el host terminó de forma ordenada, generando el reporte (7 tareas rechazadas, 0 eventos perdidos).

7.6. Rechazos y entradas inválidas

Con `tareas_rechazos.txt`, en las cuatro combinaciones:

- La línea con tipo inválido (`8,X,...`) se informó y se saltó.
- La tarea 4 ($C = 11$ ticks cada 5 en la Pico, $U = 2.2$) se rechazó con el motivo "no pasa la admisión en ningún nodo".
- Había tres esporádicas: cada nodo aceptó una y la tercera se rechazó.
- El resto de las tareas se asignó y se ejecutó normalmente.

7.7. Modo manual

Con el sistema en marcha se agregó la tarea `a 9 P 300 20 20 1`. El host la admitió en la Pico ($C = \lceil 300/87 \rceil = 4$, $U = 0.2$) sin detener nada. En el reporte aparece con 429 liberaciones y 1 713 ticks ejecutados, es decir, exactamente 4 ticks por trabajo, sin plazos perdidos.

7.8. Resumen de verificación

Requisito	Evidencia
Calendarizador local RMS y EDF	Gantt de la figura 4; 0 plazos perdidos en todo lo admitido.
Calibración en cada ejecución	s_n medido en cada corrida (41–43 y 86–87, tabla 1).
Admisión y heurísticas	Tablas 7 y pruebas `make test_planner` (13/13).
Rechazos con motivo	Tarea 5 con RMS; `tareas_rechazos.txt`.
Modo manual	Tarea 9 admitida y ejecutada sin detener el sistema.
Esporádicas, intervalo mínimo y latencia	Tabla de latencias; pulsaciones ignoradas contadas.
Plazo perdido y LED	`tareas_miss_esporadica.txt`: 3 plazos perdidos registrados.
Caída de nodos y reubicación	Dos escenarios con desconexión física del cable.
Protocolo con CRC y tramas corruptas	Prueba de trama corrupta con `fake_host.py` (`NACK`).
Desfase entre nodos	1–3 ms al inicio; tabla 5.
Terminación ordenada	Tecla `q` y caída de todos los nodos: reporte generado, recursos liberados.

8. Compilación, carga y pruebas

8.1. Host

Requiere Linux con GCC, Make y Python 3. Desde la raíz del repositorio:

```
make                                # compila host_app
make run                            # ejecutar (q para terminar)
```

Antes de ejecutar se revisan los puertos con `ls /dev/ttyACM* /dev/ttyUSB*` y se ajustan en `host.conf`. El usuario debe tener permiso sobre los puertos seriales (grupo `dialout` en Ubuntu o `uucp` en Arch).

8.2. Nodo Raspberry Pi Pico W

Con VS Code y la extensión *Raspberry Pi Pico* (SDK 2.3.1):

1. Abrir la carpeta `firmware/pico` e importarla con *Import Project*.
2. *Compile Project* genera `build/pico_firmware.uf2`.
3. *Run Project* lo carga por USB. La primera vez se conecta la placa con el botón BOOTSEL presionado.

8.3. Nodo ESP32

Con VS Code y la extensión *PlatformIO IDE*: abrir la carpeta `firmware/esp32`, compilar (*Build*) y cargar (*Upload*). Si la carga se queda en "Connecting", se mantiene presionado el botón BOOT de la placa.

8.4. Pruebas sin hardware

```
make test                       # host con dos nodos simulados (normal y caida)
```

```
make test_planner && ./test_planner   # planificador: FF/WF, RMS/EDF, rechazos, caida
make test_rts && ./test_rts     # calendarizador local en la PC
make pc_node && ./pc_node /tmp/ttyNODO   # firmware de la Pico corriendo en Linux
python3 tools/fake_host.py /dev/ttyACM0 [tick_ms] [RMS|EDF]   # probar un nodo solo
```

8.5. Experimentos y figuras

```
sh tools/correr_experimentos.sh     # 4 combinaciones -> resultados/
sh tools/figuras_doc.sh             # figuras de Gantt -> doc/figuras/
```

9. Atributo: Trabajo en Equipo

Funciona de manera efectiva como individuo y como miembro o líder en equipos diversos e inclusivos y en entornos multidisciplinarios, cara a cara, remotos y distribuidos.

a) Estrategias para el trabajo individual y en equipo de forma equitativa e inclusiva en las etapas del proyecto (planificación, ejecución y evaluación)

Planificación. El trabajo se dividió en cuatro partes de tamaño similar, una por integrante, según las herramientas y la experiencia de cada uno: José Andrés Solano se encargó del nodo ESP32, José Pablo Fernández del nodo Raspberry Pi Pico W, y Jarod de la O y Darío Garro del host. Antes de empezar se acordaron entre todos las reglas de comunicación entre el host y los nodos (el formato del protocolo), de modo que cada integrante supiera exactamente qué debía recibir y entregar su parte.

Ejecución. Cada integrante trabajó su parte de forma independiente, en su propia rama de GitHub que nace de `develop`, y la integraba a `develop` al terminar cada funcionalidad, actualizando con `pull` antes de cada `push`. Para que nadie dependiera del avance de otro, se usaron un host falso y nodos simulados que respetaban el protocolo acordado.

Evaluación. Cada integrante probó su parte antes de integrarla, y la integración final con las dos placas y el host se hizo en conjunto, de forma que todos participaron en verificar el sistema completo.

b) Planificación del trabajo mediante la identificación de roles, metas y reglas

Al inicio del proyecto se asignaron las tareas según las herramientas de cada estudiante: José Solano, que tiene una ESP32, se encargó del desarrollo de ese nodo, y de la misma manera José Fernández se encargó de la Pico. Jarod y Darío, que usan principalmente Linux, se encargaron del host. José Solano propuso un conjunto de reglas generales de comunicación entre el host y los nodos, y los demás compañeros se guiaron por ellas para trabajar por su cuenta y asegurar la comunicación correcta entre las partes. Como meta, cada integrante debía entregar su parte antes del lunes para hacer la integración el martes.

c) Acciones que promueven la colaboración entre los miembros del equipo

La creación de un grupo de WhatsApp y de un servidor de Discord fue crucial para la comunicación del equipo. Las reglas de comunicación definidas por José Solano, y el seguimiento que les dieron los compañeros, facilitaron la integración de todas las partes una vez terminadas.

d) Ejecución de las estrategias planificadas para el logro de los objetivos

Para mantener el desarrollo en paralelo, los integrantes acordaron usar un "host falso" que simulaba las salidas esperadas del host, de forma que el desarrollo de los nodos pudiera continuar mientras se construía el host real. De igual forma, los desarrolladores del host trabajaron con respuestas simuladas de los nodos, manteniendo las mismas entradas y salidas como estándar. Cada integrante se encargó de mantener actualizado su ambiente y, cuando se presentaron problemas de *merge*, los integrantes involucrados trabajaron juntos para resolver los conflictos entre ramas.

e) Evaluación del desempeño del trabajo individual y en equipo

Individual. Cada integrante comprobó que su parte funcionara antes de integrarla, con pruebas propias contra el protocolo acordado: los nodos con el host falso, y el host con nodos simulados y sus pruebas automáticas. Además, se verificó que cada parte se entregara en la fecha acordada y siguiera las reglas de comunicación.

En equipo. El desempeño del equipo se evaluó con la integración: el sistema completo, con el host y las dos placas, debía calibrar, asignar y ejecutar las tareas, y reproducir los experimentos pedidos. Los problemas que aparecieron al integrar se resolvieron entre los integrantes de las partes involucradas.

f) Evaluación de las estrategias utilizadas de equidad e inclusión

La distribución resultó equitativa: cada integrante tuvo una parte de tamaño y complejidad similares (un nodo cada uno para dos integrantes y el host para los otros dos), y todos participaron tanto en el desarrollo como en la integración. Definir las reglas de comunicación al inicio permitió que todos trabajaran con la misma información y que ninguna parte dependiera de otra para avanzar; los hosts y nodos simulados ayudaron en lo mismo. Como área de mejora, el proyecto pudo haberse empezado antes, pero por la carga del semestre no fue posible.

g) Evaluación de las acciones de colaboración entre los miembros del equipo

Los acuerdos sobre las entradas y salidas esperadas fueron una de las principales razones por las que los integrantes pudieron trabajar por su cuenta, sin esperar la implementación de otras partes. El equipo aprovechó bien los canales de llamadas y de texto de Discord, así como los mensajes de WhatsApp para avisar a los demás cuando algo estaba listo. Si el equipo pudiera mejorar en algo, sería en empezar el proyecto antes.

10. Conclusiones

- Separar el calendarizador del código de cada placa permitió usar el mismo código en dos arquitecturas distintas y probarlo en la PC antes de tener el hardware.
- EDF aprovecha mejor los nodos que RMS: con la misma carga, EDF admitió todas las tareas y llevó la Pico al 100 % sin perder plazos, mientras que RMS tuvo que rechazar una tarea por su límite de utilización más conservador. Si RMS ejecuta una carga que pasa ese límite, pierde plazos que EDF sí cumple.
- First-Fit concentra la carga en el nodo más rápido y deja capacidad libre en el otro; Worst-Fit la reparte y deja ambos nodos con margen parecido, lo que también influye en qué se puede reubicar si un nodo cae.
- La prueba de admisión por utilización garantiza los plazos solo cuando $D = T$; con plazos menores al período se pueden perder trabajos de tareas admitidas.
- La capacidad medida no depende solo de la frecuencia del procesador: la Pico (125 MHz) resultó más rápida por unidad de trabajo que el ESP32 (240 MHz), lo que confirma la importancia de calibrar en cada ejecución.

Referencias

[1] C. L. Liu y J. W. Layland, "Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.

[2] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3.ª ed. Springer, 2011.

[3] R. I. Davis y A. Burns, "A Survey of Hard Real-Time Scheduling for Multiprocessor Systems," *ACM Computing Surveys*, vol. 43, no. 4, 2011.

[4] Raspberry Pi Ltd., "The C/C++ SDK" (documentación de la Raspberry Pi Pico). `https://www.raspberrypi.com/documentation/microcontrollers/c_sdk.html`

[5] Espressif Systems, "Arduino-ESP32 Documentation." `https://docs.espressif.com/projects/arduino-esp32/en/latest/`

[6] Modbus Organization, "MODBUS over Serial Line: Specification and Implementation Guide V1.02," 2006.

[7] The Linux man-pages project, `shm_overview(7)` y `sem_overview(7)`.

A. Archivos de tareas de ejemplo

Formato: `id,tipo,W,T,D,criticidad`. Las líneas que empiezan con `#` son comentarios.

- `host/config/tasks_input.txt`: carga liviana original.
- `host/config/tareas_rms_vs_edf.txt`: experimentos RMS/EDF y FF/WF (tabla 6).
- `host/config/tareas_rechazos.txt`: tarea imposible, tercera esporádica y línea inválida.

- `host/config/tareas_miss_esporadica.txt`: esporádicas con D menor a su costo, para provocar plazos perdidos.